

K-MODU 라이브 기준 브랜치 정리 가이드

작성일 2026-08-22 · 대상 저장소 k-modu (donginjjang-eric/k-modu) · 운영 주소 <https://www.k-modu.co.kr>

이 문서의 목적

"지금 내 폴더가 운영(라이브)과 같은 상태인가?"를 확인하고, 같지 않았던 것을 바로잡은 뒤, 앞으로 같은 문제가 생기지 않게 하는 절차를 1~4단계로 정리했습니다. 명령어를 외우기보다 **왜 그 순서인지**를 이해하는 데 초점을 두었습니다.

0. 먼저 알아야 할 개념 5가지

브랜치 (branch)	같은 프로젝트의 "작업 줄기". 한 폴더에는 한 번에 하나의 브랜치만 펼쳐져(체크아웃) 있습니다. <code>git branch --show-current</code> 로 지금 어느 브랜치인지 확인합니다.
master	팀이 "이것이 정식 버전"이라고 약속한 기준 브랜치. 이번 문제의 핵심은 master가 운영보다 한참 뒤쳐져 있어서 기준 역할을 못 했다는 점입니다.
origin	GitHub에 있는 원격 저장소. <code>origin/master</code> 는 "GitHub에 올라가 있는 master"입니다. 로컬 master와 다를 수 있습니다.
배포 방식 (두 가지)	① GitHub 자동 배포 : <code>origin/master</code> 에 push 되면 Railway가 자동 빌드·배포합니다(2026-06-12부터). ② <code>railway up</code> : 지금 열려 있는 폴더의 내용을 그대로 올리는 CLI 배포. 둘 다 살아 있으므로 "master가 곧 운영"이어야 하고, 옛 브랜치가 펼쳐진 폴더에서 <code>railway up</code> 을 하면 운영이 그 옛 상태로 되돌아갑니다.
fast-forward	A 브랜치가 B 브랜치의 "과거"일 때, A를 B 위치로 그냥 옮기는 머지. 충돌이 날 수 없어서 가장 안전합니다. <code>--ff-only</code> 는 "fast-forward가 안 되면 차라리 실패해라"는 뜻입니다.

1. 작업 전 상황 (무엇이 문제였나)

라이브 사이트에서 실제 파일을 내려받아 각 브랜치와 한 줄씩 비교했습니다.

[운영 라이브]	k-modu.co.kr	=	codex/creator-action-center	0682b88	(8/10)	
[열려 있던 폴더]	agent/image-performance-optimization		d5d8c5f	(8/07)	← 라이브보다 82커밋 뒤	
[기준 브랜치]	master / origin/master			87e22ae	(7/10)	← 라이브보다 102커밋 뒤

비교 대상 브랜치	index.html	platform.css	site-i18n.js	auth-nav.js
codex/creator-action-center	동일*	동일	동일	동일
agent/image-performance-optimization	4줄 차이	동일	527줄 차이	87줄 차이
master	1268줄 차이	1620줄 차이	파일 없음	120줄 차이

* 라이브에만 있는 2줄은 Cloudflare가 이메일 주소를 가리려고 서빙 시 자동 삽입하는 코드(/cdn-cgi/l/email-protection)라 소스 차이가 아닙니다. 비교할 때마다 이 2줄은 무시하면 됩니다.

위험했던 지점

이 상태에서 열려 있던 폴더에서 `railway up` 을 했다면(또는 옛 master에 무언가 push 했다면) 운영이 옛 상태로 롤백되어, 8/10에 배포된 크리에이터 액션센터·정산·성과 대시보드 기능이 전부 사라졌을 것입니다.

2. 1단계 · 잔여물 정리

1 폴더에 남아 있던 미커밋 변경을 치웁니다. 이걸 안 하면 다음 단계의 브랜치 이동이 막히거나 엉뚱한 변경이 섞입니다.

```
git status --short          # 무엇이 바뀌어 있는지 먼저 본다
git checkout -- next-env.d.ts # 자동 생성 파일이라 되돌려도 안전
```

실행 결과	추적 파일 변경 0개. 미추적 파일(히어로 시안 이미지 10장, beauty-product-sheet.png, 플랜 문서, 로그)은 그대로 둬.
왜 미추적 파일은 안 건드렸나	git이 관리하지 않는 파일은 브랜치를 바꿔도 폴더에 그대로 남습니다. 단, 목표 브랜치가 같은 경로의 파일을 추적하고 있으면 체크아웃이 거부되므로 그 충돌만 미리 확인했고, 없었습니다.

원칙

브랜치를 옮기기 전엔 항상 `git status` 로 "지금 폴더가 깨끗한가"를 본다. 깨끗하지 않으면 커밋하거나, 되돌리거나, `git stash` 로 잠시 치운다.

3. 2단계 · master를 라이브와 똑같이 맞추기

2 기준 브랜치 master를 라이브 커밋(0682b88)으로 올려서, 앞으로 "master = 운영"이라는 약속을 복구합니다.

```
git checkout master
git merge --ff-only codex/creator-action-center
git push origin master
```

실행 결과	master 87e22ae → 0682b88 로 fast-forward, origin/master 에 push 완료. (push가 자동 배포를 트리거하지만 라이브와 같은 커밋이라 내용 변화는 없음) 검증: <code>git rev-parse master origin/master codex/creator-action-center</code> 세 값이 모두 0682b88... 로 동일.
왜 --ff-only 인가	master가 creator-action-center의 과거라는 것을 미리 확인(<code>git merge-base --is-ancestor</code>)했기 때문에 fast-forward가 가능했고, 만약 아니었다면 머지 커밋이 생기거나 충돌이 날 수 있으니 "안 되면 멈춰라"로 안전 장치를 건 것입니다.
왜 먼저 master부터 맞췄나	라이브 브랜치(codex/creator-action-center)는 별도 워크트리 폴더가 점유하고 있어 메인 폴더에서 직접 체크아웃할 수 없었습니다. master를 같은 커밋으로 끌어올리면 "라이브와 같은 출발점"을 누구나 master에서 얻을 수 있어 더 깔끔합니다.

4. 3단계 · 새 작업 브랜치 만들기

3 master(= 라이브)에서 작업용 브랜치를 따서, 작업 중에도 master는 항상 "운영 그대로"로 남겨둡니다.

```
git checkout -b work/20260822-from-live
```

실행 결과	브랜치 <code>work/20260822-from-live</code> 생성, 시작 커밋 0682b88 (라이브와 동일).
이름 규칙	<code>work/날짜-주제</code> . 작업 주제가 정해지면 <code>git branch -m work/20260822-주제명</code> 으로 바뀌도 됩니다.
왜 master에서 직접 작업하지 않나	작업 도중 문제가 생겨도 master는 깨끗하니 언제든지 운영 상태로 돌아갈 수 있고, 배포 직전 "master와 같은가?"로 점검할 기준이 유지됩니다.

5. 4단계 · 출발점 검증

4 "로컬이 정말 라이브와 같은가, 그리고 빌드가 되는가"를 눈으로 확인하고 나서야 작업을 시작합니다.

```
npm install      # 8/10 기준 lockfile에 맞춰 의존성 갱신
npm run build    # Next.js 16 프로덕션 빌드
```

npm install	정상 완료 (audit 경고만, 오류 없음).
npm run build	성공. 대시보드(admin/creator/designer) 전 라우트 포함 빌드 완료.

라이브 vs 로컬 파일 대조 결과

파일	결과	비고
index.html	동일	
platform.css	동일	
site-i18n.js	동일	
auth-nav.js	동일	
designers.html	동일	
beauty.html	2줄 차이	Cloudflare 이메일 보호 삽입 코드 — 무시
creators.html	2줄 차이	Cloudflare 이메일 보호 삽입 코드 — 무시

결론

현재 폴더(`work/20260822-from-live`)는 운영 라이브와 동일한 소스이며 빌드도 통과했습니다. 여기서 작업을 시작하면 됩니다.

대조에 쓴 명령 (재사용용, Git Bash)

```
for f in index.html platform.css site-i18n.js auth-nav.js; do
  curl -sL "https://www.k-modu.co.kr/$f" | tr -d '\r' > /tmp/live-$f
  tr -d '\r' < "$f" > /tmp/local-$f
  echo "$f: $(diff /tmp/live-$f /tmp/local-$f | grep -c '^[<>]') lines differ"
done
# 0 이면 동일. 2~4줄이고 내용이 cdn-cgi/email-protection 이면 무시.
```

6. 지금 상태 요약

운영 라이브	0682b88 (codex/creator-action-center)
master / origin/master	0682b88 — 라이브와 동일 (이번에 맞춤)
현재 폴더 브랜치	work/20260822-from-live @ 0682b88 — 라이브와 동일, 빌드 통과
옛 브랜치	agent/image-performance-optimization — 내용이 전부 creator-action-center에 포함돼 있어 더 이상 쓸 필요 없음
워크트리	<code>.worktrees/creator-action-center</code> (로그만 있음, 지워도 됨) <code>~/codex/worktrees/0829/K-MODE_NEW1</code> (8/08 detached, 옛 상태)
미추적 파일 (결정 필요)	assets/hero-concepts/* 10장, assets/beauty-products/beauty-product-sheet.png, docs/superpowers/plans/2026-08-07-beauty-product-matching-grid.md — 어느 브랜치에도 없음. 쓸 거면 작업 브랜치에 커밋, 아니면 정리.

7. 앞으로의 배포 체크리스트 (이 순서대로)

- `git status --short` — 폴더가 깨끗한가? (미커밋 변경 없음)
- `git branch --show-current` + `git log --oneline -1` — 지금 어느 브랜치, 어느 커밋인가?
- 작업 브랜치라면 먼저 master에 반영: `git checkout master && git merge --ff-only work/...` (ff가 안 되면 rebase 후 재시도) → `git push origin master`
- master 폴더에서 `npm run build` 통과 확인
- 배포: master push가 곧 자동 배포입니다(3번에서 이미 시작됨). 수동으로 `railway up` 을 쓸 때도 반드시 master가 체크아웃된 폴더에서
- 배포 후 위 "대조 명령"으로 라이브 == 로컬 확인 (버전 쿼리 `?v=` 도 갱신됐는지)

한 문장으로

master push = 운영 배포. 그러니 master는 항상 운영과 같게, 작업은 master에서 딴 브랜치에서, 배포는 master에서만. 이 세 가지만 지키면 "폴더·라이브·master가 따로 노는" 상황은 다시 생기지 않습니다.

8. 자주 헛갈리는 것

질문	답
라이브와 diff 했는데 2~4줄이 계속 다르다	내용이 <code>/cdn-cgi/l/email-protection</code> , <code>email-decode.min.js</code> 면 Cloudflare가 넣은 것. 소스 차이가 아니니 무시.
어떤 브랜치가 배포돼 있는지 모르겠다	라이브 HTML 하단의 <code><script src="/site-i18n.js?v=..."></code> 버전 문자열과, 위 대조 명령으로 각 브랜치(<code>git show 브랜치:파일</code>)와 비교하면 찾을 수 있다.
체크아웃이 "would be overwritten by checkout"으로 막힌다	미추적 파일이 목표 브랜치의 추적 파일과 같은 경로에 있는 것. 그 파일을 옮기거나 지우고 다시 시도.
체크아웃이 "already checked out at ..."으로 막힌다	그 브랜치를 다른 워크트리 폴더가 쓰고 있는 것. <code>git worktree list</code> 로 확인 후 <code>git worktree remove</code> 경로 하거나, 같은 커밋에서 새 브랜치를 따서 작업.
--ff-only가 실패한다	master에 상대 브랜치에 없는 커밋이 있다는 뜻. 무시하지 말고 <code>git log master..브랜치</code> , <code>git log 브랜치..master</code> 로 양쪽 차이를 본 뒤 rebase 또는 일반 merge로 처리.